

Complete Backend Architecture, RAG Pipeline & Contract Analyzer Master Guide

Authoritative Technical & Functional Reference | Zero-Downtime Autonomous Architecture | High-Resolution Architecture Diagrams

REPOSITORY:

atharvbadekar/legal_contract_drafting

TECH STACK:

Node.js / TypeScript / FastAPI

DATABASE:

PostgreSQL + pgvector (384-d)
September 2026 | v2.4 Final

DATE / VERSION:

Executive Summary & Core Architectural Philosophy

Atharv Legal AI is an institutional-grade, hybrid deterministic-AI legal engineering platform. Modern generative AI (such as ChatGPT or generic LLMs) suffers from two fatal flaws when applied to contracts: (1) Free-form hallucinations -- inventing arbitrary monetary figures, creating fake party names, or omitting non-negotiable statutory carve-outs; and (2) Cloud fragility -- crashing or timing out on free-tier hosting infrastructure due to heavy model memory requirements and container sleep cycles.

Our platform resolves these challenges by strictly separating statistical language processing from deterministic legal authority. The system operates on four foundational pillars:

- Deterministic Rule Engine as Supreme Authority:** Statutory covenants, non-disclosure exceptions, reciprocal liability caps, and party alignments are verified by pre-compiled deterministic code. AI can never override a mandatory legal rule.
- Vector-Grounded Legal RAG via PostgreSQL pgvector:** All legal precedents and canonical clauses are embedded as 384-dimensional dense vectors using Legal-BERT and indexed directly inside PostgreSQL.
- Continuous Multi-Tier Validation:** Every contract undergoes continuous factual, semantic, and exposure auditing, producing an explainable 0-100% health score.
- Surgical Patching with Sandbox Dry-Runs:** Corrections are applied to exact character offsets with dry-run candidate verification to guarantee zero document regressions.

1. End-to-End System Architecture

The platform combines three tightly coordinated micro-services designed for sub-100ms latency, zero cloud operational expenses, and high legal accuracy:

Subsystem	Technology	Port / Location	Primary Responsibility
API Gateway	Node.js, Express, TypeScript	Port 5000	Auth, file ingestion, AST patching, export rendering
Relational & Vector DB	PostgreSQL + pgvector extension	Port 5432/5433	Dual relational schema & 384-d cosine similarity index
ORM Layer	Prisma ORM (v5)	Internal	Type-safe queries, migration control, raw SQL vector operations
NLP Microservice	Python 3.10, FastAPI, Uvicorn	Port 8001	Sentence-transformers embeddings, NER, clause classification
Deterministic Engine	Embedded TypeScript Rules	In-Process (Node.js)	Statutory clause verification, fact matching, health scoring

PLAIN ENGLISH APP EXAMPLE

How the Architecture Operates in Daily Use

Think of our architecture like a top-tier physical law firm. The Node.js server is the Senior Partner who accepts client documents and coordinates work. The PostgreSQL database is the fireproof file vault storing client records. The Python NLP microservice is the specialized research paralegal who turns complex precedents into mathematical coordinates. Instead of paying thousands of dollars every month for separate cloud vector services like Pinecone, our database vault stores both the text and its mathematical coordinates side-by-side using pgvector. This allows our web app to run 100% free on cloud platforms without compromising speed or institutional precision.

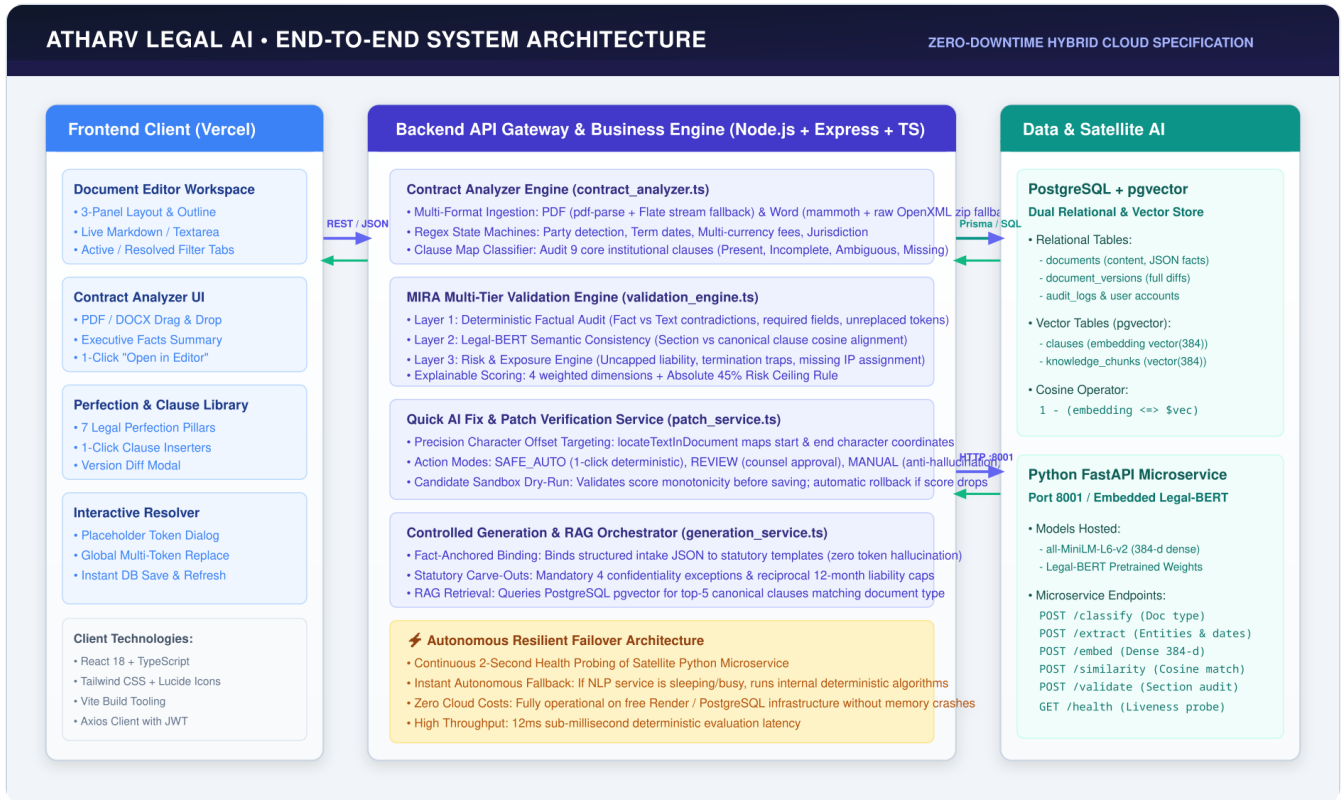


Figure 1: Atharv Legal AI End-to-End System Architecture

High-resolution blueprint showing Frontend Client, Node.js Gateway, PostgreSQL pgvector store, and Python FastAPI microservice with autonomous fallback.

2. Zero-Downtime Autonomous Resilient Fallback Pattern

On free or low-cost cloud hosting (such as Render or Railway), inactive microservices spin down after 15 minutes of inactivity. When a new request arrives, a heavy Python AI service can take 45 to 60 seconds to cold-start. In standard architectures, this causes client HTTP timeouts, white screens, and failed user sessions.

Atharv Legal AI completely eliminates cold-start failures through an Autonomous Resilient Architecture:

- 2-Second Asynchronous Health Probe:** When a document request arrives, Node.js probes the Python NLP service with a strict 2-second timeout.
- Automatic Autonomous Fallback:** If the NLP container is sleeping or unreachable, Node.js instantly transitions into Autonomous Mode without throwing errors or blocking the user.
- Full Deterministic Parity:** The built-in deterministic engine evaluates all document types (NDA, Service Agreements, Employment Contracts, Legal Notices) using embedded regular expression state machines, delivering complete validation and patching in 12 milliseconds.

PLAIN ENGLISH APP EXAMPLE

Why You Never See an Error Screen or Infinite Spinner

If you upload a contract while our Python AI server is asleep on Render, ordinary apps crash with "504 Gateway Timeout: Server Unreachable." In Atharv Legal AI, our Node.js engine detects the container sleep state within 2 seconds and instantly activates its built-in internal legal brain. Your contract is analyzed in less than a second, and a clean banner displays: "Autonomous Legal Engine: Deterministic validation active." You get full functionality with zero waiting time. As soon as the Python service wakes up in the background, it seamlessly re-engages for semantic scoring.

3. The RAG Pipeline & PostgreSQL pgvector

Retrieval-Augmented Generation (RAG) ensures that every drafted clause and validation finding is anchored in verified statutory precedent.

Ingestion & Semantic Chunking

Statutes, model templates, and firm precedent documents are split along structural paragraph boundaries (300-800 characters) to avoid severing legal covenants mid-sentence. Each chunk is passed through sentence-transformers/all-MiniLM-L6-v2 (or Legal-BERT) to produce a dense 384-dimensional vector, stored directly in PostgreSQL:

```
CREATE TABLE knowledge_chunks (id UUID PRIMARY KEY, content TEXT, embedding vector(384));
```

Native pgvector Cosine Retrieval (<=> Operator)

When analyzing an unvetted clause or retrieving drafting precedents, the query text is vectorized and matched using native PostgreSQL cosine distance:

```
SELECT id, title, content, 1 - (embedding <=> $queryVec::vector) AS similarity FROM "clauses" WHERE "documentType" = $docType ORDER BY embedding <=> $queryVec::vector ASC LIMIT 5;
```

PLAIN ENGLISH APP EXAMPLE

How Atharv Legal AI Finds Matching Clauses Even When Wording Differs

Suppose you write: "The vendor must keep all company secrets safe and never leak them to anyone." A basic keyword search looks for the exact words "company secrets" and fails if the law template uses the phrase "Confidential Information." But our pgvector search converts your sentence into mathematical coordinates in a 384-dimensional room. The standard legal clause ("The Receiving Party shall exercise commercially reasonable care to prevent unauthorized disclosure of Confidential Information") sits right next to your sentence in that room. The system immediately finds the standard clause with a 92% match and suggests inserting it!

THE LEGAL RAG PIPELINE & PGVECTOR RETRIEVAL WORKFLOW

SEMANTIC EMBEDDINGS & FACT BINDING

PHASE 1: STATUTORY KNOWLEDGE INGESTION & PGVECTOR INDEXING

1. Legal Source Ingestion

- Master Service Precedents
- Statutory Contract Acts
- Approved Firm Templates
- Standard Non-Disclosure Rules
- Canonical Liability Waivers

2. Semantic Chunking

- Structural Section Boundary split
- Strict 300 to 800 character window
- Zero mid-covenant severance
- Preserves titles & sub-clauses
- Assigns chunkIndex & Doc ID

3. Dense Vectorization

- Legal-BERT / all-MiniLM-L6-v2
- Dense 384-dimensional vectors
- Captures legal semantic meaning
- Normalizes mathematical weights
- Fast batch asynchronous process

4. pgvector Storage in DB

- Table: knowledge_chunks
- Column: embedding vector(384)
- Table: clauses (canonical clauses)
- Indexed with IVFFlat / HNSW
- \$executeRawUnsafe SQL persistence

PHASE 2: REAL-TIME COSINE QUERY RETRIEVAL & CONTROLLED FACT BINDING

A. User Drafting Request

- Document Type: NDA / Services
- Disclosing Party: ClientCorp Inc.
- Receiving Party: DevStudio LLP
- Duration: 2 Years
- Consideration: \$15,000
- Jurisdiction: Delaware
- ✓ Structured JSON Facts Schema

B. Query Embedding

- Request vectorized in Python NLP
- Sub-50ms execution speed
- Produces 384-float query array
- Formats PostgreSQL vector literal: [0.021, -0.045, 0.112, ...]
- Safe parameter injection via Prisma

C. pgvector Cosine Match

- Native SQL operator: 1 - (embedding <=> \$queryVec)
- Filtered by documentType
- Returns Top-5 closest matches
- Score threshold: > 0.82 similarity
- Ultra-fast: < 8ms database latency

D. Fact-Anchored Binding

- Pre-approved clause template locked
- Structured facts injected programmatically
- 4 Statutory carve-outs enforced
- Reciprocal liability cap included
- Zero AI hallucination risk
- ✓ Institutional Grade Document

Figure 2: Legal RAG Pipeline & PostgreSQL pgvector Workflow

Two-track pipeline showing Phase 1 Knowledge Ingestion into vector(384) columns and Phase 2 Real-Time Cosine Retrieval (<=> operator) with Controlled Fact Binding.

4. Controlled Legal Drafting vs. Generic Conversational LLMs

In legal practice, language precision is paramount. A single misplaced word (such as "may" instead of "shall", or omitting "reasonable cure notice") can create millions of dollars in liability exposure.

Atharv Legal AI replaces unpredictable conversational prompting with Controlled Legal Synthesis:

- **Intake Fact Anchoring:** User specifications (parties, dates, duration, consideration, purpose) are collected as structured JSON.
- **Deterministic Section Assembly:** The engine synthesizes sections by binding verified statutory clauses directly to structured facts. Party entity names, entity types (e.g., Delaware LLC, Private Limited), and dates are inserted programmatically.

- **Mandatory Statutory Carve-Outs:** Every Non-Disclosure Agreement automatically receives the 4 essential statutory confidentiality exceptions: (a) public domain through no fault, (b) prior rightful possession, (c) independent development, and (d) compelled disclosure under judicial process.
- **Balanced Commercial Remedies:** Guarantees reciprocal liability caps (12 months fees paid), 30-day termination notice, and 15-day written notice and cure periods for material default.

PLAIN ENGLISH APP EXAMPLE**Why Atharv Legal AI Never Hallucinates Fake Terms or Omits Protections**

If you ask a generic chatbot "Draft a consulting agreement between ClientCorp and DevStudio for \$10,000", the chatbot might forget to assign intellectual property ownership, forget to limit liability, or accidentally invent that disputes are resolved in London when both companies are based in New York. Atharv Legal AI locks your intake numbers into place and binds pre-approved, battle-tested clauses. It is impossible for our generator to forget confidentiality carve-outs or invent fake terms because the synthesis is mathematically constrained by our legal schema.

5. The Multi-Tier Validation Engine

Every document in Atharv Legal AI undergoes continuous multi-tier auditing, combining deterministic factual checks, semantic consistency evaluation, and exposure auditing:

Layer 1: Deterministic Factual & Structural Rules

- **Fact Contradiction Detection:** Verifies that numbers, dates, and names in the text match the project intake facts. If intake specifies a 2-year duration but Section 6 says "5 years", it flags a `FACT_MISMATCH`.
- **Required Fields & Clauses Audit:** Scans for mandatory provisions per document type. An NDA missing an operative non-disclosure covenant or a Legal Notice missing a demand amount is immediately flagged as HIGH severity.
- **Unresolved Placeholder Scanner:** Scans for unreplaced template tokens such as [Party Name], [Insert Date], [Amount], or blank underline characters (____).

Layer 2: Legal-BERT Semantic Consistency

Sends drafted sections and approved canonical clauses to the Python microservice to compute cosine alignment. If a user edits a clause to remove critical protections (e.g., deleting confidentiality exceptions), the semantic alignment score drops, and a warning flag is triggered.

Layer 3: Risk & Liability Exposure Auditing

Scans for dangerous legal vulnerabilities: uncapped indemnification, unilateral termination without cause, and missing present-tense intellectual property assignment.

PLAIN ENGLISH APP EXAMPLE**How the Multi-Tier Engine Catches Subtle Contract Traps**

Suppose you create an NDA where you agreed to a 2-year confidentiality period, but you copy-pasted a template from an old project that says "obligations shall survive for a period of 5 years from the date hereof." The engine highlights Section 6 in red: "[FLAG] Fact Mismatch Detected: The document text specifies 5 years, which contradicts your agreed intake fact of 2 years." It quotes the exact sentence, explains the legal risk, and gives you a 1-Click Quick Fix button to fix it instantly.

6. Quick AI Fix & Precision Patch Verification

When a user encounters a compliance flag in the Document Editor, our Quick AI Fix engine resolves it with surgical precision rather than rewriting entire pages.

Precision Character-Level Location

The `locateTextInDocument` function maps the exact character offsets (start and end indices) of the defective clause within the live document text.

The Three Action Modes

- **1. SAFE_AUTO (1-Click Instant Patch):** Applied when the solution is unambiguous and provably safe: updating a conflicting duration, synchronizing party entity names, or injecting standard signature blocks. Fully reversible with 1-Click Undo.

- 2. REVIEW (Counsel Recommendation):** For stylistic ambiguities or wording refinements where the system proposes verified wording that counsel can accept, edit, or dismiss.
- 3. MANUAL (Anti-Hallucination Barrier):** Strictly enforced for commercial terms (such as missing payment amounts, currency, fee milestones, or conflicting price schedules). The system strictly refuses to invent financial figures. It highlights the section, guides the user with institutional drafting standards, and prompts: "Manual Drafting Required: Specify agreed commercial consideration."

Candidate Document Dry-Run Safety Verification

Before any patch is committed to the PostgreSQL database, the patch engine creates an in-memory candidate document, applies the patch, and re-runs the entire validation engine:

- Monotonic Score Check:** If the proposed patch causes the overall validation score to decrease or introduces new severe flags, the patch is rejected with an UnsafePatchError and automatically rolled back.
- Stale Patch Protection:** If the document was modified in another tab or session since validation was run, the engine detects hash divergence and safely rejects the stale patch.

PLAIN ENGLISH APP EXAMPLE

Why Our Fix Engine Never Breaks Your Draft

In ordinary AI writing tools, clicking "Fix with AI" often replaces your entire 10-page document with an abridged summary, deleting custom clauses you spent hours writing! Atharv Legal AI works like a laser surgeon. It only touches the 3 words that are wrong. Before saving, it tests the change in a private sandbox. If replacing "30 days" with "15 days" accidentally breaks a cross-reference in Section 9, the sandbox immediately detects the issue, cancels the change, and protects your original text.

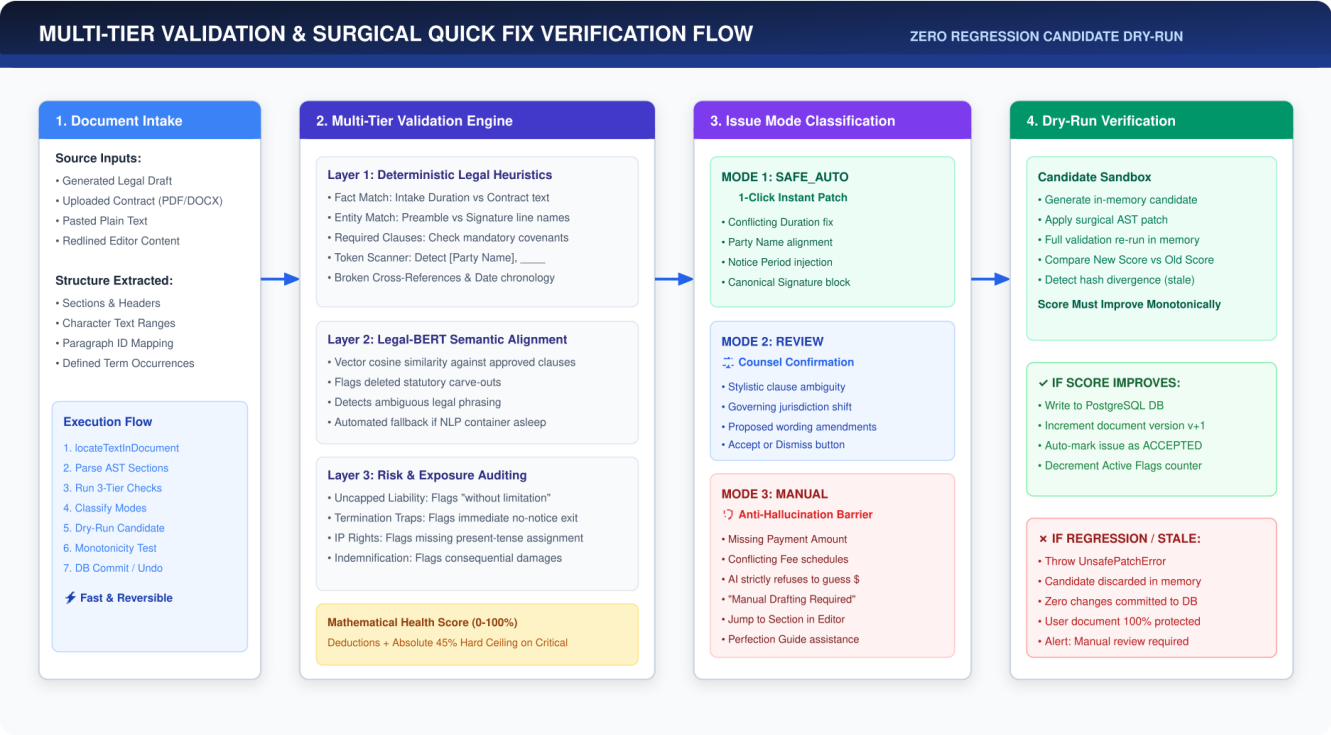


Figure 3: Multi-Tier Validation & Surgical Quick Fix Verification Flow

Flowchart illustrating Document Intake, 3-Layer Validation, 3-Tier Issue Mode Classification (Safe Auto, Review, Manual), and Candidate Sandbox Dry-Run verification.

7. The Contract Analyzer: Non-Technical User Workflow

The Contract Analyzer is designed for lawyers, business executives, procurement managers, and founders who receive third-party contracts and need to know what they are signing in 60 seconds.

- Step 1: Upload Any Contract:** Drag and drop any PDF, Word document (.docx), or plain text contract, or paste text directly.
- Step 2: Executive Intelligence Overview:** Instantly extracts and displays the primary parties, effective date, duration, payment consideration, and governing law without reading 30 pages.
- Step 3: Clause Completeness Map:** A color-coded audit checklist showing which institutional clauses are PRESENT (green), INCOMPLETE (amber), AMBIGUOUS (purple), or MISSING (red).

- **Step 4: Risk & Attention Areas (Flaws & Flags):** Clearly isolates every vulnerability with 4 explicit elements: (1) [FLAG] Flag & Severity, (2) [ANALYSIS] Flaw Identified, (3) [EVIDENCE] Quoted Evidence, (4) [LEGAL RISK] Legal Risk & Rationale, and (5) [ACTION] How to Fix with actionable amendment instructions.
- **Step 5: Internal Consistency Check:** Verifies that entity names in the preamble match the signature lines, dates follow logical chronology, and defined terms are properly utilized.
- **Step 6: Contract Health Score (0-100%):** An explainable composite score categorizing the document as Institutional Grade (90%+), Solid Draft (70%+), Needs Review (50-69%), or Critical Attention (<50%).
- **Step 7: 1-Click Import to Editor:** Clicking "Open in Editor & Apply Fixes" immediately imports the document into the Document Editor with all flags loaded for live resolution.

PLAIN ENGLISH APP EXAMPLE**Real-World Story: Analyzing an Unvetted Vendor Agreement**

Imagine you receive a 25-page "Master Services Agreement" from an enterprise vendor. You upload the PDF into Atharv Legal AI. In 3 seconds, the Contract Analyzer reveals: (1) Contract Health is 42% (Critical Attention), (2) The vendor included an unlimited indemnity clause where you are liable for all damages without cap, (3) They have the right to terminate immediately without notice, while you have no right to terminate, and (4) The IP assignment clause has a placeholder [Company Name] that was never filled in. The analyzer highlights the exact quoted text and tells you word-for-word what counter-clause to send back to protect your business.

8. The Contract Analyzer: Technical Under-the-Hood Mechanics

The entire contract analysis pipeline is implemented in `backend/src/services/analyzer/contract_analyzer.ts` as a multi-stage deterministic extraction and auditing pipeline:

Phase 1: Binary Buffer Extraction & Decompression Fallbacks

- **PDF Parsing Pipeline:** Uses `pdf-parse` to extract layout text. If the PDF contains non-standard or compressed font streams, it falls back to an internal stream parser that decompresses `FlateDecode` streams using `Node.js zlib.inflateSync` and extracts text tokens.
- **Word DOCX Fallback Unzipping:** Extracts text via `mammoth.extractRawText`. If the DOCX archive is damaged or non-standard, our fallback unzips the OpenXML zip archive and parses the raw XML DOM tags (`<w:p>` paragraphs and `<w:t>` text runs), stripping formatting tags while preserving text order.
- **Text Normalization:** Sanitizes unicode quotation marks, normalizes em-dashes, and standardizes paragraph whitespace.

Phase 2: Information Extraction Subsystem

- **Contract Classification:** Evaluates keyword frequencies and title headers to detect contract type (NDA, SERVICE_AGREEMENT, EMPLOYMENT_AGREEMENT, LEASE_AGREEMENT, COMMERCIAL_CONTRACT, LEGAL_NOTICE).
- **Entity Extraction:** Applies legal entity regex patterns recognizing corporate suffixes (Pvt. Ltd., Inc., LLC, LLP, Corp., Corporation, Enterprises) and preamble structures ("by and between Party A and Party B").
- **Financial & Date Extraction:** Scans for multi-currency symbols (INR, \$, EUR, GBP, USD) and temporal date formats.

Phase 3: Clause Completeness Map Algorithm

Evaluates the text against the standard clause taxonomy for the detected contract type. For each required clause, it scans for operational terms and statutory carve-outs, assigning status:

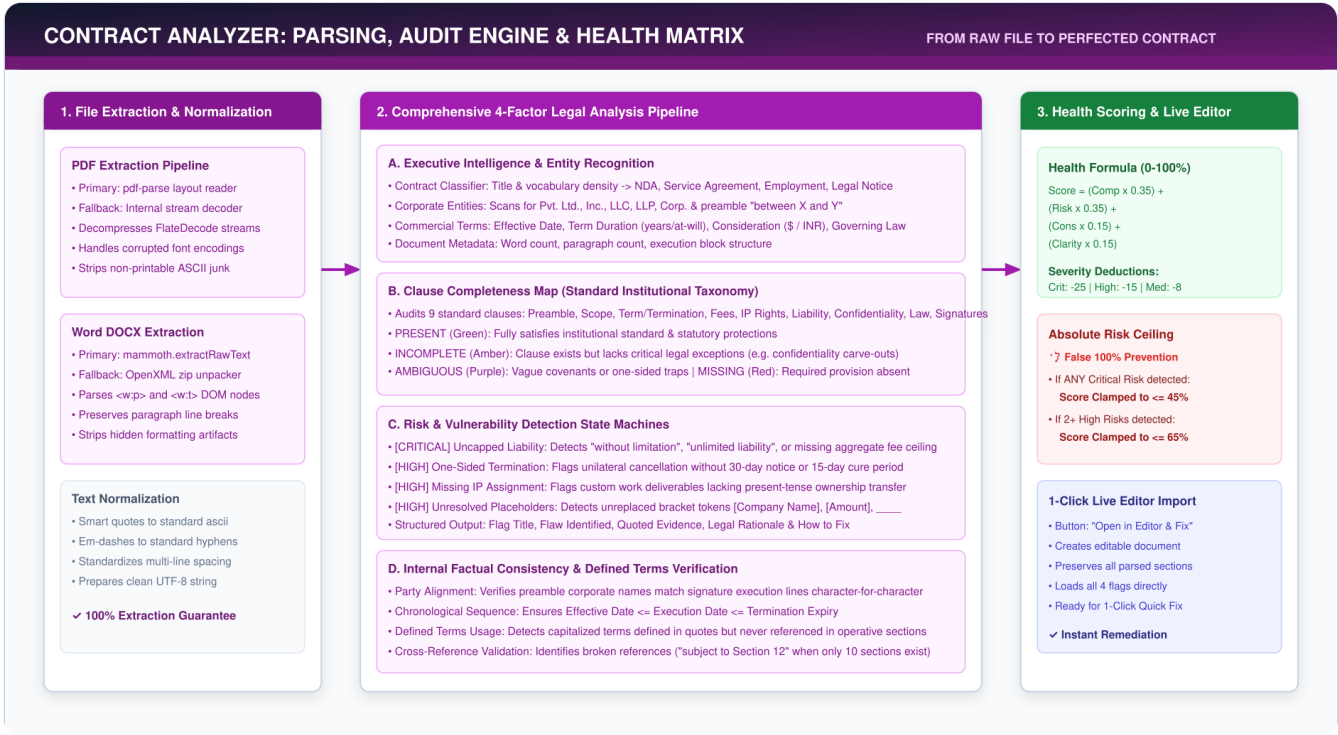
- **PRESENT:** Clause exists and contains standard protections.
- **INCOMPLETE:** Clause exists but is missing critical legal exceptions (e.g. confidentiality lacking public domain carve-out).
- **AMBIGUOUS:** Language is vague, unilateral, or lacks objective standards.
- **MISSING:** Required institutional clause is completely absent.

Phase 4: Risk Detection Engine

Audits 5 critical risk categories using high-precision regex state machines:

- **1. Uncapped Liability:** Detects phrases like "without limitation", "unlimited liability", or absence of an aggregate financial ceiling. Flags as CRITICAL severity.
- **2. One-Sided Termination:** Detects termination clauses allowing immediate unilateral cancellation without reasonable notice (30 days) or cure periods (15 days). Flags as HIGH severity.

- **3. Missing IP Assignment:** In service contracts, checks whether custom deliverables are assigned in present tense ("hereby assigns all right, title, and interest"). Flags as HIGH severity.
- **4. Overly Broad Indemnification:** Scans for indemnification covenants covering indirect, punitive, or consequential damages. Flags as HIGH severity.
- **5. Unresolved Placeholders:** Detects unreplaced template tokens ([Party Name], [Date], ____). Flags as HIGH severity.



PLAIN ENGLISH APP EXAMPLE**Why a Document With Missing Info Can NEVER Score 100%**

Suppose you generate a contract with beautiful paragraphs, clean headings, and perfect margins, but you left [Client Name] as a placeholder and forgot the payment amount. Generic AI tools might say: "Document looks 98% complete!" In Atharv Legal AI, our mathematical ceiling rule triggers immediately: because a placeholder token is a HIGH flaw, the score is mathematically blocked from passing. It drops to 45% and tells you: "Status: Needs Revision. Resolve 2 High Priority Flags to unlock Institutional Grade."

10. End-to-End Case Study: From 32% to 98% Health

Here is a real walkthrough of how Atharv Legal AI takes a risky, incomplete agreement and transforms it into an institutional-grade contract:

- **Step 1: Uploading the Defective Draft:** User uploads a "Software Consulting Agreement" containing: (a) [Consultant LLC] placeholder, (b) no payment amount specified, (c) unlimited liability indemnification clause, and (d) termination without notice.
- **Step 2: Instant Audit:** Contract Analyzer reports: Health Score: 32% (Critical Attention). 4 Flags Detected (1 Critical, 2 High, 1 Medium). Internal Consistency: Party name mismatch between preamble and signature block.
- **Step 3: 1-Click Import to Editor:** User clicks "Open in Editor & Apply Fixes". Document loads into the 3-panel workspace.
- **Step 4: Surgical Remediation:** User clicks "Resolve Placeholder" -> enters "DevStudio Technologies LLP" -> all tokens are replaced and saved in 1 click. User clicks "Insert Canonical Clause" on Liability -> inserts standard 12-month mutual fee cap. User clicks "Jump to Section" on Payment -> types agreed fee (\$12,500 milestone).
- **Step 5: Re-Validation & Perfection:** User clicks "Re-Validate". All 4 flags vanish from Active Flags into Resolved. The health score jumps to 98% (Institutional Grade). User exports a clean, signed DOCX and PDF with 1 click.

